# 04 - 합의 엔진 시스템

## 개요

합의 엔진(Consensus Engine)은 Geth에서 블록체인의 합의 메커니즘을 담당하는 핵심 컴포넌트입니다. 이 시스템은 블록의 유효성을 검증하고, 새로운 블록을 생성하며, 네트워크 전체가 동일한 상태에 합의할 수 있도록 보장합니다.

Geth는 다양한 합의 알고리즘을 지원하며, 현재 주로 사용되는 것은 Proof of Stake(PoS) 기반의 Beacon 체인 합의와 테스트넷에서 사용되는 Proof of Authority(PoA) 기반의 Clique 합의입니다.

## 핵심 구조체 및 인터페이스

### consensus.Engine 인터페이스

모든 합의 엔진은 `consensus.Engine` 인터페이스를 구현해야 합니다:

```
type Engine interface {
// 블록 생성자 주소 반환
Author(header *types.Header) (common.Address, error)
// 단일 헤더 검증
VerifyHeader(chain ChainHeaderReader, header *types.Header) error
// 배치 헤더 검증 (비동기)
VerifyHeaders(chain ChainHeaderReader, headers []*types.Header) (chan<- struct{}
// 엉클 블록 검증
VerifyUncles(chain ChainReader, block *types.Block) error
```

```
// 블록 헤더 준비
Prepare(chain ChainHeaderReader, header *types.Header) error
// 블록 완료 (상태 수정)
Finalize(chain ChainHeaderReader, header *types.Header, state vm.StateDB, body *
// 블록 완료 및 조립
FinalizeAndAssemble(chain ChainHeaderReader, header *types.Header, state *state.
body *types.Body, receipts []*types.Receipt) (*types.Block, error)
// 블록 봉인 (마이닝)
Seal(chain ChainHeaderReader, block *types.Block, results  chan<- *types.Block,
// 봉인 전 해시 계산
SealHash(header *types.Header) common.Hash
// 난이도 계산
CalcDifficulty(chain ChainHeaderReader, time  uint64, parent *types.Header) *big
// 엔진 종료
Close() error
}
```

## ChainHeaderReader 인터페이스

합의 엔진이 블록체인 데이터에 접근하기 위한 인터페이스입니다:

```
type ChainHeaderReader interface {
Config() *params.ChainConfig
```

```
CurrentHeader() *types.Header

GetHeader(hash common.Hash, number  uint64) *types.Header

GetHeaderByNumber(number  uint64) *types.Header

GetHeaderByHash(hash common.Hash) *types.Header

}
```

## 주요 합의 엔진 구현체

### 1. Beacon 합의 엔진 (Proof of Stake)

현재 이더리움 메인넷에서 사용되는 PoS 합의 메커니즘입니다.

**구조체 정의**

```
type Beacon struct {

ethone consensus.Engine // 이전 합의 엔진 (ethash/clique)

}
```

**주요 상수**

```
var (

beaconDifficulty = common.Big0  // PoS에서의 기본 난이도 (0)

beaconNonce = types.EncodeNonce(0) // PoS에서의 기본 논스 (0)

)
```

### 2. Clique 합의 엔진 (Proof of Authority)

테스트넷과 프라이빗 네트워크에서 사용되는 PoA 합의 메커니즘입니다.

### 구조체 정의

```
type Clique struct {
config *params.CliqueConfig
db ethdb.Database
recents *lru.ARCCache // 최근 블록 서명 캐시
signatures *lru.ARCCache // 서명 캐시
proposals  map[common.Address]bool  // 현재 투표 제안
signer common.Address // 서명자 주소
signFn SignerFn // 서명 함수
lock sync.RWMutex // 동시성 제어
}
```

## 주요 프로세스

### 1. 블록 검증 프로세스 (VerifyHeader)

#### Beacon 엔진의 헤더 검증

```
func (beacon *Beacon) VerifyHeader(chain consensus.ChainHeaderReader, header *ty
// 부모 블록 가져오기
parent := chain.GetHeader(header.ParentHash, header.Number.Uint64()-1)
if  parent == nil {
```

```
return consensus.ErrUnknownAncestor
}
// PoS에서 PoW로의 역전환 방지
if parent.Difficulty.Sign() == 0 && header.Difficulty.Sign() > 0 {
return consensus.ErrInvalidTerminalBlock
}
// 난이도에 따른 검증 방식 선택
if header.Difficulty.Sign() > 0 {
return beacon.ethone.VerifyHeader(chain, header) // 이전 합의 엔진 사용
}
return beacon.verifyHeader(chain, header, parent) // PoS 검증
}
```

### PoS 전용 헤더 검증

```
func (beacon *Beacon) verifyHeader(chain consensus.ChainHeaderReader, header, pa
// Extra 데이터 크기 검증
if len(header.Extra) > int(params.MaximumExtraDataSize) {
return fmt.Errorf("extra-data longer than 32 bytes (%d)", len(header.Extra))
}
// PoS 필수 필드 검증
if header.Nonce != beaconNonce {
return errInvalidNonce
```

```
}
if  header.UncleHash != types.EmptyUncleHash {
return  errInvalidUncleHash
}
// 타임스탬프 검증
if  header.Time <= parent.Time {
return  errInvalidTimestamp
}
// 난이도 검증 (PoS에서는 0이어야 함)
if  beaconDifficulty.Cmp(header.Difficulty) != 0 {
return  fmt.Errorf("invalid difficulty: have %v, want %v", header.Difficulty, be
}
// 가스 한도 검증
if  header.GasLimit > params.MaxGasLimit {
return  fmt.Errorf("invalid gasLimit: have %v, max %v", header.GasLimit, params.
}
// 블록 번호 검증
if  diff := new(big.Int).Sub(header.Number, parent.Number);  diff.Cmp(common.Big
return  consensus.ErrInvalidNumber
}
// EIP-1559 검증
if  err := eip1559.VerifyEIP1559Header(chain.Config(), parent, header);  err !=
return  err
```

```go
}
// Shanghai 업그레이드 관련 검증
shanghai := chain.Config().IsShanghai(header.Number, header.Time)
if  shanghai && header.WithdrawalsHash == nil {
return  errors.New("missing withdrawalsHash")
}
// Cancun 업그레이드 관련 검증
cancun := chain.Config().IsCancun(header.Number, header.Time)
if  cancun {
if  header.ParentBeaconRoot == nil {
return  errors.New("header is missing beaconRoot")
}
if  err := eip4844.VerifyEIP4844Header(chain.Config(), parent, header);  err !=
return  err
}
}
return  nil
}
```

## 2. 배치 헤더 검증 (VerifyHeaders)

여러 헤더를 동시에 검증하는 비동기 프로세스입니다:

```go
func (beacon *Beacon) VerifyHeaders(chain consensus.ChainHeaderReader, headers [
```

```go
// PoW와 PoS 헤더 분리
preHeaders, postHeaders := beacon.splitHeaders(headers)
if len(postHeaders) == 0 {
// 모두 PoW 헤더인 경우
return beacon.ethone.VerifyHeaders(chain, headers)
}
if len(preHeaders) == 0 {
// 모두 PoS 헤더인 경우
return beacon.verifyHeaders(chain, headers, nil)
}
// 혼합된 경우 - 별도 처리
var (
abort = make(chan struct{})
results = make(chan error, len(headers))
)
go func() {
// 비동기 검증 로직
var (
old, new, out = 0, len(preHeaders), 0
errors = make([]error, len(headers))
done = make([]bool, len(headers))
oldDone, oldResult = beacon.ethone.VerifyHeaders(chain, preHeaders)
newDone, newResult = beacon.verifyHeaders(chain, postHeaders, preHeaders[len(pre
```

```
)
// 결과 수집 및 전송
for {
for  ;  done[out];  out++ {
results <- errors[out]
if  out == len(headers)-1 {
return
}
}
select {
case  err := <-oldResult:
if !done[old] {
errors[old], done[old] = err, true
}
old++
case  err := <-newResult:
errors[new], done[new] = err, true
new++
case <-abort:
close(oldDone)
close(newDone)
return
}
```

```
}
}()
return  abort, results
}
```

### 3. 블록 준비 프로세스 (Prepare)

새로운 블록을 생성하기 전에 헤더를 준비하는 과정입니다:

```
func (beacon *Beacon) Prepare(chain consensus.ChainHeaderReader, header *types.H
// PoS 전환 여부 확인
if !isPostMerge(chain.Config(), header.Number.Uint64(), header.Time) {
return  beacon.ethone.Prepare(chain, header)
}
// PoS에서는 난이도를 0으로 설정
header.Difficulty = beaconDifficulty
return  nil
}
```

### 4. 블록 완료 프로세스 (Finalize)

트랜잭션 실행 후 블록을 완료하는 과정입니다:

```
func (beacon *Beacon) Finalize(chain consensus.ChainHeaderReader, header *types.
```

```
if !beacon.IsPoSHeader(header) {
beacon.ethone.Finalize(chain, header, state, body)
return
}
// 출금(Withdrawal) 처리
for _, w := range body.Withdrawals {
// gwei를 wei로 변환
amount := new(uint256.Int).SetUint64(w.Amount)
amount = amount.Mul(amount, uint256.NewInt(params.GWei))
state.AddBalance(w.Address, amount, tracing.BalanceIncreaseWithdrawal)
}
// PoS에서는 블록 보상이 없음 (합의 레이어에서 처리)
}
```

## 5. 블록 완료 및 조립 (FinalizeAndAssemble)

최종 블록을 조립하는 과정입니다:

```
func (beacon *Beacon) FinalizeAndAssemble(chain consensus.ChainHeaderReader, hea
state *state.StateDB, body *types.Body, receipts []*types.Receipt) (*types.Block
if !beacon.IsPoSHeader(header) {
return beacon.ethone.FinalizeAndAssemble(chain, header, state, body, receipts)
}
// Shanghai 업그레이드 처리
```

```go
shanghai := chain.Config().IsShanghai(header.Number, header.Time)
if shanghai {
if body.Withdrawals == nil {
body.Withdrawals = make([]*types.Withdrawal, 0)
}
} else {
if len(body.Withdrawals) > 0 {
return nil, errors.New("withdrawals set before Shanghai activation")
}
}
// 블록 완료
beacon.Finalize(chain, header, state, body)
// 최종 상태 루트 설정
header.Root = state.IntermediateRoot(true)
// 블록 조립
block := types.NewBlock(header, body, receipts, trie.NewStackTrie(nil))
// Verkle 트리 증명 생성 (해당하는 경우)
if chain.Config().IsVerkle(header.Number, header.Time) {
keys := state.AccessEvents().Keys()
parent := chain.GetHeaderByNumber(header.Number.Uint64() - 1)
if parent == nil {
return nil, fmt.Errorf("nil parent header for block %d", header.Number)
}
```

```go
preTrie, err := state.Database().OpenTrie(parent.Root)
if err != nil {
return nil, fmt.Errorf("error opening pre-state tree root: %w", err)
}
postTrie := state.GetTrie()
if postTrie == nil {
return nil, errors.New("post-state tree is not available")
}
vktPreTrie, okpre := preTrie.(*trie.VerkleTrie)
vktPostTrie, okpost := postTrie.(*trie.VerkleTrie)
if okpre && okpost && len(keys) > 0 {
verkleProof, stateDiff, err := vktPreTrie.Proof(vktPostTrie, keys)
if err != nil {
return nil, fmt.Errorf("error generating verkle proof for block %d: %w", header
}
block = block.WithWitness(&types.ExecutionWitness{
StateDiff: stateDiff,
VerkleProof: verkleProof,
})
}
}
return block, nil
```

```
}
```

## EIP 구현 및 검증

### EIP-1559 (Base Fee) 검증

```
func VerifyEIP1559Header(config *params.ChainConfig, parent, header *types.Head
// 가스 한도 검증
parentGasLimit := parent.GasLimit
if !config.IsLondon(parent.Number) {
parentGasLimit = parent.GasLimit * config.ElasticityMultiplier()
}
if err := misc.VerifyGaslimit(parentGasLimit, header.GasLimit); err != nil {
return err
}
// Base Fee 존재 검증
if header.BaseFee == nil {
return errors.New("header is missing baseFee")
}
// Base Fee 정확성 검증
expectedBaseFee := CalcBaseFee(config, parent)
if header.BaseFee.Cmp(expectedBaseFee) != 0 {
return fmt.Errorf("invalid baseFee: have %s, want %s, parentBaseFee %s, parentG
```

```
header.BaseFee, expectedBaseFee, parent.BaseFee, parent.GasUsed)
}
return nil
}
```

## EIP-4844 (Blob Transactions) 검증

```
func VerifyEIP4844Header(config *params.ChainConfig, parent, header *types.Head
bcfg := latestBlobConfig(config, header.Time)
if bcfg == nil {
panic("called before EIP-4844 is active")
}
// 필수 필드 존재 검증
if header.ExcessBlobGas == nil {
return errors.New("header is missing excessBlobGas")
}
if header.BlobGasUsed == nil {
return errors.New("header is missing blobGasUsed")
}
// Blob 가스 사용량 검증
if *header.BlobGasUsed > bcfg.maxBlobGas() {
return fmt.Errorf("blob gas used %d exceeds maximum allowance %d",
*header.BlobGasUsed, bcfg.maxBlobGas())
```

```
}

// Excess Blob Gas 정확성 검증

expectedExcessBlobGas := CalcExcessBlobGas(config, parent, header.Time)

if *header.ExcessBlobGas != expectedExcessBlobGas {

return  fmt.Errorf("invalid excessBlobGas: have %d, want %d",

*header.ExcessBlobGas, expectedExcessBlobGas)

}

return  nil

}
```

# 에러 처리

## 주요 에러 타입

```
var (

// 알려지지 않은 조상 블록

ErrUnknownAncestor = errors.New("unknown ancestor")

// 가지치기된 조상 블록

ErrPrunedAncestor = errors.New("pruned ancestor")

// 미래 블록

ErrFutureBlock = errors.New("block in the future")

// 잘못된 블록 번호

ErrInvalidNumber = errors.New("invalid block number")
```

```
// 잘못된 터미널 블록
ErrInvalidTerminalBlock = errors.New("invalid terminal block")
)
```

### Beacon 엔진 전용 에러

```
var (
errTooManyUncles = errors.New("too many uncles")
errInvalidNonce = errors.New("invalid nonce")
errInvalidUncleHash = errors.New("invalid uncle hash")
errInvalidTimestamp = errors.New("invalid timestamp")
)
```

## 최적화 및 성능

### 1. 서명 캐시

Clique 엔진에서는 서명 검증 성능을 위해 LRU 캐시를 사용합니다:

```
type sigLRU struct {
*lru.ARCCache
}

func  ecrecover(header *types.Header, sigcache *sigLRU) (common.Address, error)
```

```
hash := header.Hash()

if address, known := sigcache.Get(hash); known {

return address, nil // 캐시된 결과 반환

}

// 서명 복구 수행

// ...

}
```

### 2. 비동기 검증

`VerifyHeaders` 메서드는 여러 헤더를 동시에 검증하여 성능을 향상시킵니다:

- 고루틴을 사용한 병렬 처리
- 채널을 통한 비동기 결과 전달
- 중단 신호 지원

### 3. 스냅샷 캐시

Clique 엔진은 투표 상태를 스냅샷으로 캐시하여 빠른 검증을 지원합니다.

## 디버깅 및 모니터링

### 로깅 포인트

```
// 헤더 검증 실패 시

log.Warn("Invalid header", "number", header.Number, "hash", header.Hash(), "err"
```

```
// 합의 전환 감지 시
log.Info("Consensus transition detected", "block", header.Number, "difficulty",

// 출금 처리 시
log.Debug("Processing withdrawal", "address", w.Address, "amount", w.Amount)
```

### 메트릭 수집

- 헤더 검증 시간
- 블록 완료 시간
- 에러 발생 빈도
- 캐시 히트율

## 관련 컴포넌트

### 의존성 관계

- **core.BlockChain**: 블록체인 상태 관리
- **core.StateProcessor**: 트랜잭션 처리
- **miner.Worker**: 블록 생성
- **params.ChainConfig**: 네트워크 설정

### 상호작용 방식

1. **블록 검증**: `BlockChain.insertChain()` → `Engine.VerifyHeader()`
2. **블록 생성**: `Worker.generateWork()` → `Engine.Prepare()` → `Engine.FinalizeAndAssemble()`

3. **합의 전환**: 설정에 따른 동적 엔진 전환

합의 엔진 시스템은 Geth의 핵심 보안 메커니즘으로, 네트워크의 무결성과 일관성을 보장하는 중요한 역할을 담당합니다.